

1. What does it mean for a graph to be connected?
2. Define a disconnected graph with an example.
3. What is a connected component?
4. How can you determine if a graph is connected using DFS or BFS?
5. What is the difference between strongly and weakly connected in directed graphs?
6. What is a bridge in a graph?
7. Is a tree a connected graph? Justify.
8. What is a k -connected graph?
9. What is a cut vertex?
10. Define Eulerian trail and Eulerian circuit.
11. Give an example of an Eulerian graph.

State Euler's Theorem for undirected graphs.

Answer:

Euler's Theorem:

A connected undirected graph has an Eulerian circuit if and only if all vertices have even degree.

Prove that a connected graph with all even-degree vertices has an Eulerian circuit.

Answer:

Proof Idea:

- Start from any vertex.
- Always follow an unused edge, avoiding bridges unless necessary.
- Since all degrees are even, you'll always be able to return.
- This ensures a circuit exists using all edges once.

What is Fleury's Algorithm?

Answer:

Fleury's Algorithm is a step-by-step method to find an Eulerian trail or circuit in a graph, provided one exists.

Avoid using bridges (edges whose removal disconnects the graph) until necessary.

What are the basic steps in Fleury's Algorithm?

Answer:

- 1. Check if the graph has an Eulerian trail or circuit.**
- 2. Choose a starting vertex:**
 - If circuit: any vertex**
 - If trail: one of the two odd-degree vertices**
- 3. Follow edges one by one:**
 - Do not cross a bridge unless it is the only edge left.**
- 4. Mark used edges to avoid repetition.**
- 5. Stop when all edges have been used.**

Q51. How does Fleury's Algorithm avoid bridges?

Answer:

Before using an edge, check if its removal would disconnect the remaining graph. If so, skip it unless no alternative is available.

Q52. How to detect if an edge is a bridge during execution?

Answer:

Temporarily remove the edge and perform DFS to check connectivity between its endpoints. If they become disconnected, it's a bridge.

When should Fleury's Algorithm stop?

Answer:

When all edges of the graph have been used exactly once.

Q57. What is a drawback of Fleury's Algorithm?

Answer:

Its high time complexity due to repeated bridge checking makes it inefficient for large graphs.

What is a Hamiltonian cycle?

Answer:

A Hamiltonian cycle is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex.

Q60. What is a Hamiltonian path?

Answer:

A Hamiltonian path is a path that visits every vertex exactly once, but does not need to return to the starting point.

Q73. What is the Chinese Postman Problem (CPP)?

Answer:

The Chinese Postman Problem seeks the shortest closed path (circuit) that visits every edge of a graph at least once.

Goal: Minimize total cost or distance.

Q74. Why is it called the Chinese Postman Problem?

Answer:

It was introduced by Chinese mathematician Meigu Guan in 1962, hence the name.

Q75. When does the Chinese Postman Problem reduce to finding an Eulerian circuit?

Answer:

If the graph is Eulerian (all vertices have even degree), then the CPP solution = Eulerian circuit, since it already traverses every edge once.

Q76. What is the strategy to solve CPP in non-Eulerian graphs?

Answer:

1. Identify odd-degree vertices.
 2. Find shortest paths between all odd-degree vertex pairs.
 3. Find a minimum weight matching among them.
 4. Duplicate the edges along those paths.
 5. Find an Eulerian circuit in the modified graph.
-

Q87. What is the Traveling Salesman Problem (TSP)?

Answer:

TSP asks:

"Given a list of cities and distances between them, what is the shortest possible route that visits each city exactly once and returns to the origin city?"

Q88. Is TSP a Hamiltonian cycle problem?

Answer:

Yes. TSP is a weighted version of the Hamiltonian cycle problem.

- **TSP seeks a minimum-cost Hamiltonian cycle in a weighted complete graph.**
-

Q89. What is the objective of the TSP?

Answer:

Minimize the total cost/distance/time required to visit every vertex (city) exactly once and return to the start.

Q90. Is TSP NP-hard?

Answer:

Yes.

TSP is an NP-hard and NP-complete problem, meaning:

- **No known polynomial-time algorithm.**
 - **Exponential time for exact solutions.**
-

Q91. What are the real-world applications of TSP?

Answer:

- **Logistics & delivery routing**
 - **Circuit board manufacturing**
 - **Ride-sharing and taxi routing**
 - **DNA sequencing**
-

Q92. What is the brute-force time complexity for TSP?

Answer:

**$O(n!)$, where n = number of cities.
It checks all possible permutations.**

Q93. What is the Held-Karp algorithm?

Answer:

An exact dynamic programming algorithm for TSP with time complexity $O(n^2 \cdot 2^n)$.

Much faster than brute force but still exponential.
